

Desafío I: Sweet Crush

EMILIANO DE JESUS LINCE DIAZ

Facultad de Ing. Electrónica UdeA

Correo: dejesus.lince@udea.edu.co

ESTEBAN GARCES ATEHORTUA

Facultad de Ing. Electrónica UdeA

Correo: esteban.garcas2@udea.edu.co

I. CONTEXTUALIZACIÓN Y ANALISIS DEL PROBLEMA

El desafío exige implementar un juego llamado (Sweet Crush) en lenguaje C++ en el framework Qt. El juego consiste en fichas que están representadas exactamente por 3 bits, empaquetados de forma continua en memoria dinámica, sin poder usar struct, class, template, STL ni string. Esto obliga a resolver el problema de dos maneras: una con acceso a bits (indexación y operaciones bitwise) y la segunda de lógica de juego (combinaciones, cascadas, redimensionamiento), ambas independientes de la interfaz gráfica.

Analizando el reto, identificamos los siguientes retos principales de desarrollo del juego:

- Leer/escribir 3 bits que pueden estar repartidos entre dos bytes consecutivos.
- Redimensionar la memoria (sin sobredimensionar), incluso cuando se inserte/elimine filas o columnas en posiciones intermedias.
- Procesar las cascadas simultaneas sin duplicar eliminaciones.

II. DISEÑO DE LA INTERFAZ DE USUARIO

II-A. Para todo menú

Todo menú debe incluir verificación de los datos ingresados (osea hacer validación), una opción de ayuda para comandos validos, y una opción de salir del juego (que siempre va estar disponible).

II-A1. Menú de inicio del juego:

- **Determinar el tamaño del tablero:** Pedir que ingrese cantidad de filas y cantidad de columnas por separado.

II-A2. Menú en partida:

- **Eliminar ficha:** Pedir al usuario las coordenadas en posición de la ficha a eliminar en la fila y columna correspondiente de dicha ficha.
- **Agregar fila:** Pedir al usuario la fila a agregar.
- **Eliminar fila:** Pedir al usuario la fila a eliminar.
- **Agregar columna:** Pedir al usuario la columna a agregar.
- **Eliminar columna:** pedir al usuario la columna a eliminar.

Se imprimirá en pantalla el tablero de juego, con su correspondiente representación grafica de cada ficha de juego y en formato binario.

III. DISEÑO DE LA CODIFICACIÓN DE BITS

III-A. Representación binaria de las fichas

Tabla I
REPRESENTACIÓN DE LAS FICHAS

Ficha	Representación bits	Tipo	R. gráfica
1	001	Normal	β
2	010	Normal	Ω
3	011	Normal	Σ
4	100	Normal	Γ
5	101	Normal	θ
6	110	Normal	μ
7	111	Especial	Estado Eliminación
8	000	Especial	Estado vacío

III-B. Definición de los estados especiales

La ficha 8 que en su representación de bits es (000) representa el estado especial (vacío). La ficha 7 que en su representación de bits es (111) representa el estado especial (Eliminación).

- **Ficha 8:** Definimos que esta ficha va representar el estado (vacío), como una representación de que no hay fichas en esa posición.
- **Ficha 7:** Definimos que esta ficha va representar el estado (Eliminación), como una confirmación de que la ficha en la posición que está recorriendo se debe eliminar.

4. DISEÑO DE LAS FÓRMULAS DE ACCESO A BITS

Dada una posición (fila, columna) en un tablero de Filas $\times$ Columnas:

Tabla II
FÓRMULAS DE ACCESO A NIVEL DE BITS

Paso	Fórmula	Propósito
Índice lineal de la ficha	$I = \text{fila} \times \text{Columnas} + \text{columna}$	Convierte coordenadas 2D en una posición lineal.
Bit inicial de la ficha	$B = I \times 3$	Ubica el primer bit de la ficha en la tira continua.
Byte que la contiene	$\text{Byte} = \lfloor B/8 \rfloor$	Ubica el byte de memoria donde inicia la ficha.
Desplazamiento en el byte	$\text{Offset} = B \pmod{8}$	Bits de desplazamiento dentro de ese byte.
Bytes totales a reservar	$\text{Bytes} = \lceil ((3 \times \text{Filas} \times \text{Columnas}) + 7)/8 \rceil$	Redondeo hacia arriba (evita pérdida por truncamiento).

Offset puede tomar cualquier valor entre 0 y 7 (no solo 0-5). Sin embargo, solo cuando Offset está entre 0 y 5 la ficha cabe completa dentro de un único byte.

Cuando Offset es 6 o 7, los 3 bits de la ficha quedan repartidos entre el byte actual y el siguiente, y deben tratarse como un caso adicional.

- **Offset 0-5 (caso simple):** desplazar el byte a la derecha (offset) bits, aplicar máscara 00000111 (valor 7) con AND para leer; para escribir, invertir la máscara desplazada a la posición del offset con NOT, aplicar AND con el byte original para limpiar los 3 bits, y luego OR con la ficha nueva ya desplazada al offset.
- **Offset 6-7 (caso de cruce entre bytes):** leer los bits que quedan disponibles al final del byte actual y combinarlos, mediante desplazamientos y OR, con los bits que faltan al inicio del byte siguiente; para escribir, aplicar el mismo procedimiento de máscara/limpieza en ambos bytes por separado, sin alterar los bits vecinos de ninguno de los dos.

IV. MÓDULOS PARA FUNCIONES

Para el código vamos a usar 4 módulos, cada uno con una función diferente a los demás:

- **Módulo 1 (Motor de bits):** En este módulo se realizarán todas las operaciones matemáticas y de bits, solo se usará para leer y escribir en la memoria las fichas. Este módulo tendrá las siguientes funciones:
 - **Función calcular Byte Desplazamiento:** Dada la fila y la columna se encargará de calcular en qué byte de la memoria está la ficha y qué desplazamiento tiene.
 - **Función leer ficha:** Extrae de la memoria los 3 bits de la ficha teniendo en cuenta los 3 casos posibles que se pueden dar.
 - **Función escribir ficha:** Modifica los bits de la ficha en la memoria sin modificar ningún otro bit vecino.
- **Módulo 2 (Gestor de memoria):** Este módulo se encarga del manejo de la memoria para que el juego funcione correctamente y respete la regla del 65 %.
 - **Función inicializar tablero:** Calcula los bytes necesarios para el tablero y pide la reserva de la memoria.
 - **Función expandir memoria:** Solo se llamará cuando se agrega una fila o columna, reservará la nueva memoria necesaria o liberará la vieja.
 - **Función reducir memoria:** Se encarga de revisar si la memoria cae por debajo del 65 % luego de que se elimina una fila o columna, y liberará memoria si es necesario.
- **Módulo 3 (Lógica del juego):** Este módulo se encarga de las reglas del juego:

- **Función generar fichas aleatorias:** Llenará las posiciones vacías del tablero con fichas al azar de manera uniforme.
- **Función buscar combinaciones:** Recorre las filas y columnas del tablero buscando las posibles combinaciones de 3 o más fichas.
- **Función eliminación por combinación:** Elimina las fichas validadas y aplicará la gravedad para que caigan nuevas fichas a esa posición.
- **Función cascadas:** Detecta si al caer las nuevas fichas se generaron nuevas combinaciones.
- **Función eliminación por usuario:** Recibe la coordenada dada por el usuario y elimina esa ficha, también activa la gravedad para nuevas fichas.
- **Función detectar cruces simultáneos:** Detecta si generan combinaciones tipo T, L, etc. Las elimina y activa gravedad para fichas nuevas.
- **Función gravedad:** Después de cada eliminación se activa para rellenar los espacios vacíos del tablero.
- **Funciones de agregar/eliminar fila/columna:** Se encargan de reorganizar el tablero luego de la elección del usuario haciendo los movimientos de bits necesarios.
- **Módulo 4 (Gráficos en consola):** Este módulo se encarga del manejo de los tableros para mostrarlos en consola:
 - **Función tablero fichas:** Muestra el tablero de juego con las fichas, especial para el jugador.
 - **Función tablero binario:** Muestra el tablero de juego en binario, especial para el profesor.
 - **Función puntuación:** Muestra la puntuación junto con los movimientos y cascadas realizadas.

V. ESTADO DEL JUEGO

Se representa mediante variables individuales de tipo primitivo (no struct, según la restricción del desafío): filas actuales, columnas actuales, bytes reservados, eliminaciones realizadas por el usuario, fichas eliminadas en total, combinaciones detectadas, cascadas de la eliminación actual y puntuación acumulada. Estas variables se pasan entre funciones por puntero/referencia según se requiera actualizarlas.

VI. PENDIENTES DE DISEÑO ANTES DE IMPLEMENTAR

- Detallar el procedimiento exacto de compactación de bits al reducir memoria (qué bloque de bits se copia y hacia dónde, al eliminar una fila/columna intermedia).
- Definir el criterio de puntuación.

- Validar con casos de prueba el manejo del offset 6 y 7 descrito en la sección 4, por ser el punto de mayor riesgo técnico del proyecto.



UNIVERSIDAD
DE ANTIOQUIA

1 8 0 3